

1. Datos primitivos del lenguaje Java.

- byte
- short
- int
- long
- float
- double
- boolean
- char

VARIABLES DE TIPOS PRIMITIVOS.					
Nombre	Tipo	Tamaño	Valor por defecto	Forma de inicializar	Range
Boolean	Lógico	1 bit	False	Boolean a=true	True-false
Char	Carácter	16 bits	Null	Char a="Z"	Unicode
Byte	Numero entero	8 bits	0	Byte a =0	-128 a 127
Short	Numero entero	16 bits	0	Short a =12	-32.768 a 32.767
Int	Numero entero	32 bit	0	Int a= 1250	-2.147.483.648 a 2.147.483.649
Long	Numero entero	64 bits	0	Long a= 125000	-9*10 ¹⁸ a 9*10 ¹⁸
Float	Numero real	32 bits	0	Float a =3.1	-3,4*10 ³⁸ a 3,4*10 ³⁸
Double	Numero real	64 bits	0	Double a = 125.2333	-1,79*10 ³⁰⁸ a 1,79*10 ³⁰⁸

2. pseudocódigo.

```
int a = 0;
int b = 0;
int c = 0;
c = a++ + b++;
¿Cual es el resultado de la siguiente operación?
c= 0;
```

3. Sentencias de control de flujo.

Las declaraciones dentro de los archivos generalmente son ejecutadas de arriba hacia abajo, es decir en el orden en que van apareciendo. Las declaraciones de control de flujo , sin embargo , rompen precisamente ese flujo de ejecución empleando la toma de decisión, el bucle y la bifurcación.

Dentro del control de flujo existen :

Toma de decisiones : (if-then, if-then-else, switch).

bucle (for, while, do-while)

bifurcación (break, continue, return)

4. Niveles de acceso a una clase Java.

- public
- private
- protected
- No modifier

La siguiente imagen muestra los niveles de acceso de los elementos de las clases con los distintos modificadores

Modificadores de acceso				
	La misma clase	Otra clase del mismo paquete	Subclase de otro paquete	Otra clase de otro paquete
public	X	X	X	X
protected	X	X	X	
default	X	X		
private	X			

5. Anotaciones.

- Son una forma de [metadatos que proveen información al compilador](#), proporcionan datos sobre un programa que no forma parte del programa en sí. Las anotaciones no tienen un efecto directo en el funcionamiento del código que anotan.

6. Contexto.

- Área de almacenamiento donde se encuentran instancias

Para los tipos de contexto jboss controla todas esas zonas de memoria, ejemplos de contexto son 7:

- Singleton
- Prototype
- Request
- Session
- Global session
- Application
- Websocket

De ellos, los 5 últimos sólo están disponibles en una aplicación web-aware.

[Para Java Platform, Enterprise Edition: The Java EE Tutorial](#)

Table 23-1 Scopes

Scope	Annotation	Duration
Request	@RequestScoped	A user's interaction with a web application in a single HTTP request.
Session	@SessionScoped	A user's interaction with a web application across multiple HTTP requests.
Application	@ApplicationScoped	Shared state across all users' interactions with a web application.
Dependent	@Dependent	The default scope if none is specified; it means that an object exists to serve exactly one client (bean) and has the same lifecycle as that client (bean).
Conversation	@ConversationScoped	A user's interaction with a servlet, including JavaServer Faces applications. The conversation scope exists within developer-controlled boundaries that extend it across multiple requests for long-running conversations. All long-running conversations are scoped to a particular HTTP servlet session and may not cross session boundaries.

7. Managed beans.

-Bean administrado por el servidor, los primeros mb existieron en el servidor tomcat

Managed Bean es una clase regular Bean de java registrada con JSF. En otras palabras es un bean de java administrado por el framework JSF. Managed Bean contiene los métodos getter y Setter , lógica de negocios, o eventualmente como backing bean (un bean que contiene todo los valores del formulario HTML).

Managed beans trabaja como modelo para los componentes UI. Un managed bean puede ser accedido desde una página JSF .

8. Tipos de diagramas

Los diferentes tipos de diagramas se dividen en **dos grupos**, los que **describen la estructura** del sistema y los que **describen su comportamiento**. Dentro de los de comportamiento hay a su vez un subgrupo con los de **interacción**.

- Diagramas de secuencia: diagrama dinámico que muestra el intercambio de mensajes entre las clases u objetos.

Diagrama estática o estructurales

- **Diagrama de clases:** Describe los diferentes tipos de objetos en un sistema y las relaciones existentes entre ellos. Dentro de las clases muestra las propiedades y operaciones, así como las restricciones de las conexiones entre objetos.
- **Diagrama de componentes:** Muestra la jerarquía y relaciones entre componentes de un sistema software
- **Diagrama de objetos:**(También llamado Diagrama de instancias) Foto de los objetos en un sistema en un momento del tiempo.
- **Diagrama de despliegue:**Muestra la relación entre componentes o subsistemas software y el hardware donde se despliega o instala.

- **Diagrama de paquetes:** Muestra la estructura y dependencia entre paquetes, los cuales permiten agrupar elementos (no solamente clases) para la descripción de grandes sistemas.
- **Diagrama de estructura compuesta:** Descompone jerárquicamente una clase mostrando su estructura interna.

Diagrama dinámico o de comportamiento.

- **Diagrama de actividades:** Describe la lógica de un procedimiento, un proceso de negocio o workflow
- **Diagrama de casos de uso:** Permite capturar los requerimientos funcionales de un sistema.
- **Diagrama de estados:** Permite mostrar el comportamiento de un objeto a lo largo de su vida.

Diagrama de interacción: (Subgrupo dentro de los diagramas de comportamiento): Describen cómo los grupos de objetos colaboran para producir un comportamiento. Son los siguientes:

- **Diagrama de secuencia:** Muestra los mensajes que son pasados entre objetos en un escenario.
- **Diagrama de comunicación:** Muestra las interacciones entre los participantes haciendo énfasis en la secuencia de mensajes.
- **Diagrama de tiempos:** Pone el foco en las restricciones temporales de un objeto o un conjunto de objetos.
- **Diagrama global de interacciones:** Trata de mostrar de forma conjunta diagramas de actividad y diagramas de secuencia.

9. Tipos de configuración de Jboss:

- standalone.xml
- standalone-full.xml
- standalone-ha.xml
- standalone-full-ha

standalone.xml :

EAP_HOME/standalone/configuration/standalone.xml

Este es el archivo de configuración predefinido para un servidor standalone. Contiene toda la información acerca del servidor, incluyendo los subsistemas, redes de trabajo, deployments, socket bindings y otros detalles de configuración. Esta configuración es usada automáticamente cuando inicia el servidor standalone.

Soporte para Java EE **Web-Profile** mas algunas extensiones tal como servicios Web **RestFul** y soporte para invocaciones remotas **EJB3**.

standalone-full.xml:

EAP_HOME/standalone/configuration/standalone-full.xml

Incluye soporte para todos los subsistemas posibles, excepto aquellos requeridos para alta disponibilidad. (Mensajes)

Soporte de **Java EE Full-Profile** y todas las capacidades del servidor **sin clustering**.

standalone-ha.xml:

EAP_HOME/standalone/configuration/standalone-ha.xml

Habilita todos los subsistemas predeterminados y agrega los subsistemas mod_gluster y Jgroups para un servidor standalone, para poder participar dentro de un cluster de alta disponibilidad. Este archivo no es aplicable para una administración de dominio.

Su perfil predeterminado es con capacidades de cluster (agrupamientos)

standalone-full-ha.xml:

EAP_HOME/standalone/configuration/standalone-full-ha.xml

Incluye soporte para todos los subsistemas posibles, incluidos los necesarios para una alta disponibilidad.

Perfil completo con capacidades de Clustering (agrupamiento)

La configuración para mensajero está contenido dentro el archivo

\$EAP_HOME/standalone/configuration/standalone-full.xml or

\$EAP_HOME/standalone/configuration/standalone-full-ha.xml.

The element in the server configuration file contains all JMS configuration.

10. Tipos de archivos para empaquetar aplicaciones Enterprise

- jar
- war
- ear(Este puede contener jar, war)

No hay diferencias estructurales entre los archivos; todos se archivan utilizando [compresión](#) zip / jar .

Sin embargo, están destinados a diferentes propósitos.

- [Los archivos jar](#) (archivos con extensión .jar) están destinados a contener [bibliotecas](#) genéricas de clases, recursos, archivos auxiliares, etc. de Java.
- [Los archivos War](#) (archivos con extensión .war) están diseñados para contener [aplicaciones web completas](#). En este contexto, una aplicación web se define como un único grupo de archivos, clases, recursos, archivos .jar que se pueden empaquetar y acceder como un contexto de servlet.
- [Los archivos ear](#) (archivos con extensión .ear) están destinados a contener aplicaciones [empresariales completas](#). En este contexto, una aplicación empresarial se define como una colección de archivos .jar, recursos, clases y múltiples aplicaciones web.

11. Java Server Faces

Tecnología que permite la separación entre la presentación y comportamiento en las aplicaciones Web. (Esto estaba en la diapositiva de Jesus)

(Definición 1).

Es una tecnología y un framework MVC (Modelo Vista Controlador) para aplicaciones Java basadas en web que simplifica el desarrollo de interfaces de usuario en aplicaciones Java EE. JSF usa JavaServer Pages (JSP) como la tecnología que permite hacer el despliegue de las páginas.

(Definición 2).

JSF es un framework MVC (Modelo-Vista-Controlador) basado en el API de Servlets que proporciona un conjunto de componentes en forma de etiquetas definidas en páginas XHTML mediante el framework Facelets. Facelets se define en la especificación 2 de JSF como un elemento fundamental de JSF que proporciona características de plantillas y de creación de componentes compuestos. Antes de la especificación actual se utilizaba JSP para componer las páginas JSF.

Características:

- Se trata de una tecnología que ejecuta del lado del servidor y no del lado del cliente.
- La interfaz de usuario es tratada como un conjunto de componentes UI. Este es un concepto fundamental para la comprensión de la tecnología JSF.
- Definición de las interfaces de usuario mediante **vistas** que agrupan **componentes** gráficos.
- Conexión de los componentes gráficos con los datos de la aplicación mediante los denominados **beans gestionados**.
- Conversión de datos y validación automática de la entrada del usuario.
- Navegación entre vistas.
- Internacionalización
- A partir de la especificación 2.0 un modelo estándar de comunicación Ajax entre la vista y el servidor.

12. Estándares que se utilizan para los servicios web

API JaxWs y Jax-Rs

JAX-RS -> REST-FULL -> Es una arquitectura

JAX-WS-> SOAP -> Es un protocolo

Diferencias:

	SOAP	REST
Significado	Simple Object Access Protocol	Representational State Transfer

Diseño	Es un protocolo estandarizado con reglas predefinidas por seguir.	Es un estilo de arquitectura con pautas y recomendaciones.
Enfoque.	Basado en funciones (los datos están disponibles como servicios , ejemplo (get user))	Controlado por datos (datos disponibles como recursos, ejemplo "usuario")
Sin estado (Statefulness)	Sin estado es predeterminado, pero es posible realizar una API con estado de sesión.	Sin estado (no existen sesiones del lado del servidor).
Caching	Las llamadas a la API no pueden ser almacenadas en cache,	Las llamadas a la API pueden ser almacenadas en cache.
Performance	Requiere más banda de ancha y poder de computación	Requiere pocos recursos.
Message format	Solo XML.	Texto plano, HTML, XML,JSON, YAML, y otros.
Transfer protocol(s)	HTTP, SMTP, UDP, and others.	Solo HTTP.
Seguridad	WS-Security con SSL soporte. De compilación Built-in ACID.	Soporta HTTPS y SSL.
Recomendado para	Apps empresariales, apps con alta seguridad, ambientes distribuidos, servicios financieros, servicios de telecomunicaciones.	Servicios Web para Apis públicas, servicios móviles, redes sociales.
Ventajas	Seguridad alta, estandarizado, extensibilidad.	Escalabilidad, mejor desempeño, flexibilidad, facilidad de navegación.

Desventajas	Desempeño pobre, mayor complejidad, menos flexible.	Menor seguridad, No apto para ambientes distribuidos.
	SOAP usa servicios de interfaces para exponer la funcionalidad al cliente. Dentro de SOAP , el archivo WSDL provee al cliente la información necesaria la cual puede ser utilizada para entender que servicios ofrece.	REST utiliza Uniform Service locators para acceder a los componentes sobre el dispositivo del hardware. Por ejemplo, si existe un objeto el cual representa los datos de un empleado almacenado en una URL como http://demo.guru99 , abajo están algunas de las URI que pueden existir para acceder a los servicios. http://demo.guru99.com/Employee http://demo.guru99.com/Employee/1

13. Enterprise Bean

- Se ejecuta en el servidor y encapsula la lógica de negocio de una aplicación
- Un Enterprise Java Bean (EJB) es un componente de negocio Java Enterprise, y para su ejecución necesita
- un contenedor EJB/J2EE (JBoss, WAS, OAS, etc). El hecho de usar EJB's te da acceso a los servicios del Contenedor EJB (manejo de transacciones, seguridad ,persistencia, etc) que simplifican bastante la construcción de soluciones empresariales.

14. API JMS

Es una API que permite a las aplicaciones crear, enviar, recibir y leer mensajes usando una comunicación asíncrona. Es asíncrono porque no tiene que esperar la comunicación, es de bajo acoplamiento porq no necesita respuesta

15. To_char To_date

-Para quitar las diferencias en la configuración o para el idioma

16. Intersect

-Es un operador que obtiene como resultado todos los registros que son obtenidos por ambas consultas.

El INTERSECT operador de Oracle **compara el resultado de dos consultas y devuelve**

Comentado [1]: definición de Jesús

las filas distintas que ambas consultas generan.

La siguiente declaración muestra la sintaxis del INTERSECT operador:

```
SELECT column_list_1 FROM T1
```

```
INTERSECT
```

```
SELECT column_list_2 FROM T2;
```

Al igual que el UNIÓN operador, debe seguir estas reglas cuando use el INTERSECT operador:

- El número y el orden de las columnas deben ser iguales en las dos consultas.
- El tipo de datos de las columnas correspondientes debe estar en el mismo

grupo de tipos de datos, como numérico o carácter.

17. Patrones para la implementación de la lógica de negocio.

- Business Object:

Los Business Objects son objetos que concentran toda la lógica de su aplicación. Utilice Business Objects para separar la lógica y los datos comerciales mediante un modelo de objeto.

- Composite Entity

-

El patrón Composite sirve para construir objetos complejos a partir de otros más simples y similares entre sí, gracias a la composición recursiva y a una estructura en forma de árbol

Utilizar *Composite Entity* para modelar, representar y manejar un conjunto de objetos persistentes relacionados en vez de representarlos como beans de entidad específicos individuales. Un bean *Composite Entity* representa un grupo de objetos.

Un objeto persistente es un objeto que se almacena en algún tipo de almacenamiento. Normalmente múltiples clientes comparten los objetos persistentes. Los objetos persistentes se pueden clasificar en dos tipos: objetos genéricos y objetos dependientes.

Un objeto genérico es autosuficiente. Tiene su propio ciclo de vida y maneja sus relaciones con otros objetos. Todo objeto genérico podría referenciar o contener uno o más objetos. El objeto genérico normalmente maneja el ciclo de vida de estos objetos. De ahí, que esos objetos sean conocidos como objetos dependientes. Un objeto dependiente puede ser un simple objeto auto-contenido o podría a su vez contener otros objetos dependientes.

El ciclo de vida de un objeto dependiente está fuertemente acoplado al ciclo de vida del objeto genérico. Un cliente sólo podría acceder al objeto dependiente a través del objeto genérico. Es decir, los objetos dependientes no se exponen directamente a los clientes porque su objeto padre (genérico) los maneja. Los objetos dependientes no pueden existir por sí mismos. En su lugar, siempre necesitan tener su objeto genérico (o padre) para justificar su existencia.

Normalmente, podemos ver las relaciones entre un objeto genérico y sus objetos dependientes como un árbol. El objeto genérico es la raíz del árbol (el nodo raíz). Cada objeto dependiente puede ser un objeto dependientes solitario (un nodo hoja) que es hijo

del objeto genérico. O, el objeto dependiente puede tener relaciones padre-hijo con otros objetos dependientes, en cuyo caso se considera un nodo rama.

Un **bean Composite Entity** puede representar un objeto genérico y todos sus objetos dependientes relacionados. Esta combinación de objetos persistentes relacionados dentro de un sólo bean de entidad reduce drásticamente el número de beans de entidad requeridos por la aplicación. Esto genera un bean de entidad altamente genérico que puede obtener mejor los beneficios de los beans de entidad que un bean de entidad específico.

18. Que se busca durante el diseño de las clases

-Alta cohesión

- Bajo acoplamiento

Acoplamiento

El acoplamiento es el grado en el cual una clase sabe acerca de otra clase. Si el único conocimiento que tiene una clase A sobre una clase B, es que la clase B ha expuesto a través de su interfaz, entonces la clase A y la clase B se dicen que tienen un acoplamiento bajo, lo cual es una buena idea.

Si, en otro lado, la clase A se basa en partes de la clase B que no son parte de la interfaz de la clase B, entonces el acoplamiento entre las clases es más estricta...lo cual no es buena idea. En otras palabras, si A conoce mas de lo que debería sobre la manera en la que B fué implementada, entonces A y B están acopladas estrictamente.

Ejemplo de alto acoplamiento

```
1  class DoTaxes{
2      float rate;
3      float doColorado(){
4          SalesTaxRates str = new SalesTaxRates();
5          rate = str.salesRate; // Esto debería ser la llamada a un método
6                               //rate = str.getSalesRate("CO");
7          return rate;
8      }
9  }
10
11 class SalesTaxRates{
12     public float salesRate; // debería ser private
13     public float adjustedSalesRate; // debería ser private
14
15     public float getSalesRate(String region){
16         salesRate = new DoTaxes().doColorado();
17         // Calculos
18         return adjustedSalesRate;
19     }
20 }
```

El **acoplamiento fuerte** significa que las clases relacionadas necesitan saber detalles internos unas de otras, los cambios se propagan por el sistema y el sistema es posiblemente más difícil de entender.

Por ello deberemos siempre intentar que nuestras clases tengan un **acoplamiento bajo**. **Cuantas menos cosas conozca la clase A sobre la clase B, menor será su acoplamiento.**

Lo ideal es conseguir que la clase A sólo conozca de la clase B lo necesario para que pueda hacer uso de los métodos de la clase B, pero no conozca nada acerca de cómo estos métodos o sus atributos están implementados.

Los atributos de una clase deberán ser privados y la única forma de acceder a ellos debe ser a través de los métodos getter y setter.

Un bajo acoplamiento permite:

Entender una clase sin leer otras
Cambiar una clase sin afectar a otras
Mejora la mantenibilidad del código

- **Alta cohesión.**

Definición de cohesión

La cohesión se refiere al grado en el cual una clase tiene un rol bien definido o responsabilidad individual, es decir **que tan bien está diseñada una clase** . Mientras más enfocada esté una clase , más alta será su cohesión. Este tipo de clases son más fáciles de mantener y son más reutilizables.

Una prueba fácil de cohesión consiste en examinar una clase y decidir si todo su contenido está directamente relacionado con el nombre de la clase y descrito por el mismo.

Una alta cohesión hace más fácil:

- Entender qué hace una clase o método.
- Usar nombres descriptivos.
- Reutilizar clases o métodos.
- Una alta cohesión tiene en cuenta también el nombre de la clase, el nombre tiene que estar directamente relacionado con el contenido de la misma.

Ejemplo

Un ejemplo de alta cohesión sería:

```
1 class ServicioUsuarios{
2     public int altaUsuario(String usu){}
3     public int bajaUsuarios(String usu){}
4 }
5
6 class ServiciosLibros{
7     public int altaLibros(String lib){}
8     public int bajaLibros(String lib){}
9 }
```

Ejemplo (con pseudo-código) de baja cohesión:

```
1 ServiciosUsuarios
2   altaUsuarios(usu)
3
4 ServiciosLibros
5   altaLibros(lib)
6
7 OtrosServicios
8   bajaUsuarios(usu)
9   bajaLibros(lib)
```

En el primer ejemplo los nombres de los métodos son claros y coinciden con el nombre de la clase cada uno de ellos, entonces es un **ALTA COHESIÓN**.

- .

19. Clase thread safe.

Puede ejecutarse en múltiples hilos, manteniendo el estado de cada uno.

Una función (o método) thread-safe puede ser invocada por múltiples hilos de ejecución sin preocuparnos de que los datos a los que accede dicha función (o método) sean corrompidos por alguno de los hilos ya que se asegura la atomicidad de la operación, es decir, la función se ejecuta de forma serializada sin interrupciones, por lo general, adquiriendo un mutex.

<https://www.journaldev.com/1061/thread-safety-in-java>

--

<https://www.genbeta.com/desarrollo/diferencia-entre-reentrant-y-thread-safe>

20. Etapas del proceso ADS Sigetic

- Inicio
- Análisis
- Diseño
- Pruebas
- Codificación
- Liberación.

21. Etapa de codificación SIGETIC.(Documentos que se entregan)

- Prueba unitaria
- Manual Instalaciòn
- Guías
- Cápsulas
- Centro de ayuda

22. Producto cartesiano

Ocurre cuando no se especifican todas las condiciones en la cláusula WHERE de una consulta.

23. En qué opción no se necesitan utilizar subqueries.

- A) Para definir el conjunto de filas que serán insertadas en la tabla destino
- B) para definir el conjunto de filas que serán incluidas en un vista
- C) para definir uno o más valores que serán asignados a las filas existentes
- D) para seleccionar el conjunto de filas en la tabla con un orden específico

respuesta es D

El uso de subconsultas es una técnica que permite utilizar el resultado de una tabla SELECT en otra consulta SELECT. Permite solucionar consultas complejas mediante el uso de resultados previos conseguidos a través de otra consulta. El SELECT que se coloca en el interior de otro SELECT se conoce con el término de SUBSELECT

. Ese SUBSELECT se puede colocar dentro de las cláusulas WHERE, HAVING, FROM o JOIN.

24. Ventaja de usar variables Bind

Permiten la reutilización de sentencias SQL.

25. Referencias de llave foránea

Es una columna de una tabla que es referenciada en otra tabla.

26. Transacción

- Grupo de sentencias SQL que ejecutan una unidad lógica de trabajo y cuya ejecución debe realizarse y/o deshacerse en conjunto

27. Tipo de dato Clob , Blob y varchar

(Character Large Object) puede almacenar cadenas de más de 4000 gigabytes hasta 128 terabytes de datos .

Blob (Binary large object)

Reemplaza al tipo de dato LONG RAW y almacena el contenido multimedia dentro de la base de datos. Generalmente, estos datos son imágenes, archivos de sonido y otros objetos multimedia; a veces se almacenan como códigos de binarios BLOB.

Para trabajar con BLOB se requiere de [Objetos Directorios en la base de datos](#). Los Objetos Directorios no son objetos que le pertenecen a un esquema, [todos los directorios creados son adueñados por el usuario SYS](#). Para crear directorios necesitamos el privilegio de [sistema CREATE ANY DIRECTORY](#). Estos Objetos Directorios serán una referencia a una ubicación de un directorio del sistema operativo.

Ejemplo

Para actualizar los tipos de dato BLOB, **se deberá asignarles un string indicando donde esta ubicado el archivo que se desea almacenar. Al insertar o actualizar el registro, se transfiere el contenido de dicho archivo a la base de datos.**

Por ej.si se agrega el registro en un procedimiento:

```
New
    AttCode = 1
    AttBlob = 'C:\images\Photo.jpg'
Endnew
```

Esto hace que la imagen Photo.jpg ubicada en c:\images se almacene en el registro 1.

Si por el contrario la actualización se realiza en una transacción, se digita en el atributo Blob el camino al archivo.

Varchar2

varchar2(x): almacena cadenas de caracteres de longitud variable determinada por el argumento "x" (obligatorio). Que sea una cadena de longitud variable significa que, si definimos un campo como "varchar2(10)" y almacenamos el valor "hola" (4 caracteres), Oracle solamente ocupa las 4 posiciones (4 bytes y no 10 como en el caso de "char"); por lo tanto, si la longitud es variable, es conveniente utilizar este tipo de dato y no "char", así ocupamos menos espacio de almacenamiento en disco.

- Es obligatorio especificar el tamaño.
- El tamaño máximo en BD es 4000 bytes y el mínimo 1 byte.
- El tamaño máximo en PL/SQL es 32767 bytes y el mínimo 1 byte.
- STRING y VARCHAR son sinonimos de VARCHAR2. Ver NVARCHAR2.
- Usando VARCHAR2 en lugar de CHAR ahorramos espacio de almacenamiento.
- Copiar
 - Un char(10) almacenará 'PEPE '
 - Un varchar2(10) almacenará 'PEPE'
- En contra tiene que si se escriben muchas veces hay que hacer un mayor esfuerzo de mantenimiento del sistema para mantener la eficiencia (compactar).

TIPO	CARACTERISTICAS	OBSERVACIONES
CHAR	Cadena de caracteres (alfanuméricos) de longitud fija	Entre 1 y 2000 bytes como máximo. Aunque se introduzca un valor más corto que el indicado en el tamaño, se rellenará al tamaño indicado. Es de longitud fija, siempre ocupará lo mismo, independientemente del valor que contenga
VARCHAR2	Cadena de caracteres de longitud variable	Entre 1 y 4000 bytes como máximo. El tamaño del campo dependerá del valor que contenga, es de longitud variable.
VARCHAR	Cadena de caracteres de longitud variable	En desuso, se utiliza VARCHAR2 en su lugar
NCHAR	Cadena de caracteres de longitud fija que sólo almacena caracteres Unicode	Entre 1 y 2000 bytes como máximo. El juego de caracteres del tipo de datos (datatype) NCHAR sólo puede ser AL16UTF16 ó UTF8. El juego de caracteres se especifica cuando se crea la base de datos Oracle
NVARCHAR2	Cadena de caracteres de longitud variable que sólo almacena caracteres Unicode	Entre 1 y 4000 bytes como máximo. El juego de caracteres del tipo de datos (datatype) NCHAR sólo puede ser AL16UTF16 ó UTF8. El juego de caracteres se especifica cuando se crea la base de datos Oracle
LONG	Cadena de caracteres de longitud variable	Como máximo admite hasta 2 GB (2000 MB). Los datos LONG deberán ser convertidos apropiadamente al moverse entre diversos sistemas. Este tipo de datos está obsoleto (en desuso), en su lugar se utilizan los datos de tipo LOB (CLOB,NCLOB). Oracle recomienda que se convierta el tipo de datos LONG a alguno LOB si aún se está utilizando. No se puede utilizar en cláusulas WHERE, GROUP BY, ORDER BY, CONNECT BY ni DISTINCT Una tabla sólo puede contener una columna de tipo LONG. Sólo soporta acceso secuencial.
LONG RAW	Almacenan cadenas binarias de ancho variable	Hasta 2 GB. En desuso, se sustituye por los tipos LOB.
RAW	Almacenan cadenas binarias de ancho variable	Hasta 32767 bytes. En desuso, se sustituye por los tipos LOB.
LOB (BLOB, CLOB, NCLOB, BFILE)	Permiten almacenar y manipular bloques grandes de datos no estructurados (tales como texto, imágenes, videos, sonidos, etc) en formato binario o del carácter	Admiten hasta 8 terabytes (8000 GB). Una tabla puede contener varias columnas de tipo LOB. Soportan acceso aleatorio. Las tablas con columnas de tipo LOB no pueden ser replicadas.
BLOB	Permite almacenar datos binarios no estructurados	Admiten hasta 8 terabytes
CLOB	Almacena datos de tipo carácter	Admiten hasta 8 terabytes

28. Vista Normal y vista materializada.

- Vista (vista Normal)

Una Vista es una **tabla lógica virtual creada al momento por un "select query"** , también puede decirse que es un objeto de Oracle que al consultarlo ejecuta por detrás una query y nos devuelve un resultado, el cual , **no es almacenado en ningún lugar del disco y cada vez que se accede a la vista se ejecuta la query y nos devuelve la información que en ese momento exista (en tablas, otras vistas, funciones o procedimientos).**

La vista es creada con el comando Create View. La vista puede ser creada de mas de una tabla o a partir también de vistas. Una de las características de la vista es que si se realiza una actualización dentro del contenido de la vista , este cambio se ve reflejado en la tabla original, y si cualquier cambio ha sido aplicado en la tabla original , este debería ser reflejado en la vista. Sin embargo esto hace el desempeño más lento.

Sintaxis para crear la vista

```
Create View V As <Query Expression>
```

Ejemplo:

```
CREATE VIEW V_Customer  
AS SELECT First_Name, Last_Name, Country  
FROM Customer;
```

-Una **Vista Materializada (Materialized Views)** es otro tipo de objeto de Oracle que aunque técnicamente es lo mismo que una Vista (ejecuta una consulta sql), **lo que hace es almacenar físicamente en caché (En disco) el resultado de ejecutar una query en un determinado momento**, de forma que cada vez que consultemos la Vista Materializada lo que vamos a recuperar es lo que había en el origen en el momento en el que se creó o se refresco la VM.

La **actualización podrá ser hecha manualmente o a partir de triggers o procedimientos.**, siendo que la primera carga inicial de datos se realiza cuando se define la vista. El proceso de actualización de la vista materializada es llamado mantenimiento de vista materializada (**Materialized View Maintenance**).

Aunque uno de los problemas es la **actualización de los datos** , ya que los datos en la vista pueden estar desfasados con los datos actuales de las tablas originales si es que no se ejecuta el proceso de actualización, **también se incrementa el tamaño de la base de datos porque los resultados se almacenan físicamente.**


```
CREATE MATERIALIZED VIEW nueva_vista_materializada
[TABLESPACE nuestro_tablespace]
[BUILD {IMMEDIATE | DEFERRED}]
[REFRESH {ON COMMIT | ON DEMAND | [START WITH
fecha_inicial] NEXT intervalo_tiempo } |
{COMPLETE | FAST | FORCE | NEVER} ]
[{ENABLE|DISABLE} QUERY REWRITE]
AS SELECT tabla1.campo_a, tabla2.campo_b
FROM tabla1 , tabla2
WHERE tabla1.campo_a = tabla2.campo_a
AND ...
```

Donde podemos especificar de qué forma queremos que se recuperen los datos: de forma inmediata o en otro momento:

- **BUILD IMMEDIATE:** El resultado se almacena al ejecutar la query
- **BUILD DEFERRED:** Sólo se crea la definición, el resultado se almacenará más adelante. Para ello podemos utilizar cuando queramos llenarla la función REFRESH del paquete de Oracle DBMS_MVIEW.

También podemos especificar cada cuánto tiempo o cuándo queremos que se actualicen los datos de la vista:

- **REFRESH ON COMMIT:** los datos se actualizan cada vez que se haga COMMIT sobre los objetos fuente.
- **REFRESH ON DEMAND:** sólo se actualizarán cuando se haga manualmente a través de las funciones REFRESH de Oracle (más adelante se explica cuáles son y cómo invocarlas).
- **REFRESH [START WITH fecha_inicial] NEXT intervalo_tiempo:** con esta sentencia podemos establecer una actualización periódica de la vista. La fecha_inicial puede definirse como sysdate y el intervalo_tiempo cada cuanto tiempo queremos realizar la tarea (podemos incrementar numéricamente el sysdate)

Y especificar si permitimos a Oracle o no reescribir las queries cuando en una consulta se intente acceder a las tablas origen, de forma que por detrás el dueño de Oracle ataque directamente a las vistas materializadas, cuyo acceso y recuperación de datos es más rápido, sin que el usuario necesite hacer nada. Esto sólo lo hará el Optimizador de Oracle si la consulta lo permite (que las tablas a las que va a buscar la información coinciden con las tablas origen de la vista materializada).

- **ENABLE QUERY REWRITE:** Oracle puede reescribir las queries (esto para la actualización de datos)
- **DISABLE QUERY REWRITE:** lo contrario

La ventaja de este tipo de objeto la encontramos cuando la query asociada es muy compleja, tiene numerosos joins y se utiliza con frecuencia, ya que nos permite mejorar el rendimiento de la SQL al tenerla almacenada en memoria y ejecutarse una sólo vez. Sin embargo, el principal hándicap es que nos vemos obligados a mantenerla actualizada de forma regular para no perder calidad en los datos. Para ello podemos definir en el script de creación de la Vista Materializada el tipo de

actualización que tendrá al ejecutar los métodos propios de Oracle para refrescar. **Existen los siguientes tipos de actualización para una Vista materializada**

- **FAST:** Es la opción más recomendada por su rendimiento. Es una actualización incremental, es decir, sólo actualiza los registros que hayan cambiado. Para que esta opción funcione correctamente necesitamos generar estadísticas de la vista para que Oracle esté al tanto de los cambios. La forma de hacerlo es creando un log de la vista materializada sobre la PK de cada tabla origen.

```
CREATE MATERIALIZED VIEW LOG ON tabla_origen
```

```
WITH PRIMARY KEY
```

```
INCLUDING NEW VALUES;
```

- **COMPLETE:** regenera por completo el resultado al borrar y volver a ejecutar de nuevo la query.
- **FORCE:** Es el valor por defecto. En caso de que se pueda, se ejecutará el FAST; si no, el COMPLETE.
- **NEVER:** nunca se refresca la vista.

Si modificamos el origen y queremos que éste se refleje en la Vista materializada, tenemos que invocar a una función del paquete DBMS_MVIEW de Oracle que se encarga de actualizar los datos. Tenemos dos a nuestra disposición:

- **DBMS_MVIEW.REFRESH:** con este método pasamos por parámetro la Vista Materializada para que se actualice:

```
DBMS_MVIEW.REFRESH ("VISTA_MATERIALIZADA")
```

29. Grant Read

- Es la sentencia que permite que el esquema de Base de Datos **puestos** tengan acceso de solo lectura.

30. Etapas de diseño sigetic(Documentos que se entregan)

- Diagramas de componentes y estados
- Historia de Usuario
- Mensajes y Validaciones Generales
- Reglas de negocio generales
- Documento de diagrama de componentes y estados
- Plan de pruebas
- Documento de dudas y observaciones
- Trazabilidad de requerimientos
- Casos de prueba de componentes
- Caso de prueba de integración y de negocio
- Control de revisiones y avance
- Plan de capacitación
- Paquete de diseño de la solución tecnológica
- Wireframe, prototipo o boceto
- Hojas de estilo(Archivos ,css)

- Diagrama lógico de BD
- Diagrama relacional
- Dimensionamiento de BD
- Diccionario de datos
- Modelo físico(scripts)
- Manual de Instalación de BD

31. Historias de usuario

En este documento, se describen las necesidades específicas del usuario por cada paquete de trabajo, planteando de manera detallada la funcionalidad esperada; reglas de negocio, validaciones y mensajes específicos y los elementos que se deberán probar (criterios de aceptación).

32. Descripción de roles

-Documento del SIGETIC en donde se encuentran el listado de todas las funciones para cada uno de los usuarios.

33. Pruebas unitarias

En este documento, se especifican las pruebas realizadas por los desarrolladores a cada paquete de trabajo, asegurando que el producto entregado a los probadores cumple con las funcionalidades básicas para dar inicio al periodo de pruebas.

34. Funciones del desarrollador, probador y diseñador

Desarrollador:

- Complementar / actualizar las historias de usuario con reglas de negocio, mensajes, campos y validaciones específicas.
- Diseña pruebas unitarias.
- Corrige hallazgos detectados en la verificación.
- Desarrolla o corrige componentes de la solución tecnológica.
- Ejecuta pruebas unitarias.
- Verifica y corrige hallazgos durante la ejecución de las pruebas

Probador:

- Realiza o modifica mapeo de la Base de Datos contra módulos del sistema e identifica los mensajes y validaciones.
- Diseña los casos de prueba.
- Realiza verificaciones a los documentos de la etapa de diseño.
- Ejecuta pruebas de instalación de la base de datos, actualiza la documentación de análisis y diseño, desarrolla scripts de regresión, carga y estrés y revisa el avance diario y ejecución de pruebas de componente.
- Ejecuta pruebas de sistema de la solución tecnológica, modifica los scripts para pruebas de regresión, carga y estrés, y ejecuta pruebas de regresión.
- Realiza la revisión y pruebas a materiales de capacitación

- **Diseñador (de experiencia de usuario):**

- Realiza ajustes o modificaciones al wireframe, prototipo o boceto.
- Elabora y entrega las hojas de estilo.
- Ejecuta pruebas de experiencia de usuario y apoya en el ajuste de diseño visual.
- Continúa con la ejecución de pruebas de experiencia de usuario.

- **Diseñador (instruccional):**

- Elabora materiales de capacitación.
- Ajusta y corrige los materiales de capacitación.
- Coordina ejercicio y simulacros para el manejo del proyecto o producto y capacita al CAU, Áreas solicitantes y al Usuario final.

35. Objetivo del sigetic

- Es el Sistema de Gestión de Tecnologías de la Información y Comunicaciones, el cual, define los procedimientos que regulan las actividades que deben seguirse en materia de TIC para estandarizar, controlar y hacer eficiente la gestión de los servicios en los proyectos de desarrollo e infraestructura en el Instituto.

36. Paquete de trabajo.

Nivel de componente más bajo para crear un EDT (Estructura de desglose de proyectos)/WBS(lo mismo pero en inglés).

Obtenido de una plantilla que está en SIGETIC

En este documento se listan todos los paquetes de trabajo que sean identificados a partir de los requerimientos capturados en el documento "Requerimientos del servicio". *Cada paquete de trabajo es una unidad que representa el esfuerzo o trabajo que le será asignado al personal para cumplir con el proyecto. En los paquetes de trabajo, también se deberá incluir todo el trabajo que debe ser hecho para iniciar la construcción de la aplicación pero que no son requerimientos del área usuaria, tal como: configuración de ambiente de desarrollo, configuración inicial de la aplicación, funcionalidad de acceso, desarrollo del menú, etc.*

37.- Lo del signo de (+) en base de datos.

Según Jesús , si está a la izquierda del operador es RIGHT JOIN, cuando está del lado derecho es un LEFT JOIN.

Pongo lo que habia puesto en un documento en linea.

La operador de oracle para la unión externa (outer join) es un carácter de más entre paréntesis (+).

Uniones externas a la izquierda y derecha (Left and Right join).

Para entender la diferencia de estas uniones considerar la siguiente sintaxis :

SELECT ... FROM table1, table2.

Se asume que las tablas son unidas de esta forma table1.column1 and table2.column2. También se debe suponer que la tabla1 contiene una fila con un valor nulo en la columna1. Para realizar un LEFT JOIN, la cláusula WHERE es:

WHERE table1.column1 = table2.column2 (+);

En un left join , el operador outer join está actualmente a la derecha del operador de igualdad.

Si asumimos que la tabla2 contiene una fila con un valor nulo en la columna2. Para realizar un RIGHT JOIN, se debe intercambiar la posición del operador outer join a la izquierda del operador de igualdad y donde la cláusula WHERE comienza.

WHERE table1.column1 (+) = table2.column2;

Ejemplo Left Outer Join . La siguiente consulta utiliza un left outer join, el operador outer join se coloca a la derecha del operador de igualdad.

```
SELECT p.name, pt.name
FROM products p, product_types pt
WHERE p.product_type_id = pt.product_type_id (+)
ORDER BY p.name;
```

NAME	NAME
2412: The Return	Video
Chemistry	Book
Classical Music	CD
Creative Yell	CD
From Another Planet	DVD
Modern Science	Book
My Front Line	
Pop 3	CD
Space Force 9	DVD
Supernova	Video
Tank War	Video
Z Files	Video

ejemplo de Right Outer Join

```

SELECT p.name, pt.name
FROM products p, product_types pt
WHERE p.product_type_id (+) = pt.product_type_id
ORDER BY p.name;

```

NAME	NAME
-----	-----
2412: The Return	Video
Chemistry	Book
Classical Music	CD
Creative Yell	CD
From Another Planet	DVD
Modern Science	Book
Pop 3	CD
Space Force 9	DVD
Supernova	Video
Tank War	Video
Z Files	Video
	Magazine

Limitaciones del outer join

No puedes colocar el operador outer join en ambos lados.

No se puede colocar el outer join con el operador IN (WHERE p.product_type_id (+) IN (1, 2, 3, 4)).

No se puede usar el operador outer join con el operador OR. (WHERE p.product_type_id (+) = pt.product_type_id OR p.product_type_id = 1).

38. Funciones de agregación count() y Sum();

Las funciones de agregación básicas que soportan todos los gestores de datos son las siguientes:

- COUNT: devuelve el número total de filas seleccionadas por la consulta.
- MIN: devuelve el valor mínimo del campo que especifiquemos.
- MAX: devuelve el valor máximo del campo que especifiquemos.
- SUM: suma los valores del campo que especifiquemos. Sólo se puede utilizar en columnas numéricas.
- AVG: devuelve el valor promedio del campo que especifiquemos. Sólo se puede utilizar en columnas numéricas.

39.

Definición de normalización

La normalización es la transformación de las vistas de usuario complejas y del almacén de datos a un juego de estructuras de **datos más pequeñas y estables**. Además de ser más simples y estables, las estructuras de datos son más fáciles de mantener que otras estructuras de datos. (Kendall, 2005)

Las bases de datos se normalizan para:

- Evitar la redundancia de datos
- Proteger la integridad de los datos
- Evitar problemas de actualización de los datos en las tablas

La normalización de una base de datos persigue varios objetivos, principalmente reducir la redundancia de datos y simplificar las dependencias entre columnas, aplicándose de manera acumulativa. Lo anterior quiere decir que la segunda forma normal incluye a la primera, la tercera a la segunda y así sucesivamente. Una base de datos que esté en segunda forma normal, por tanto, cumplirá las dos primeras reglas de normalización.

Reglas de normalización

Primera regla.- Primera forma Normal (1FN).

El valor de una **columna debe ser una entidad atómica, indivisible**, excluyendo así las dificultades que podría conllevar el tratamiento de un dato formado de varias partes.

1. Todos los atributos, **valores almacenados en las columnas, deben ser indivisibles**.
2. No deben existir grupos de **valores repetidos**.

Supongamos que tienes en una tabla una columna Dirección para almacenar la dirección completa, dato que se compondría del nombre de la calle, el número exterior, el número interior (puerta), el código postal, el estado y la capital.

Segunda regla

1. Crear tablas separadas para aquellos grupos de datos que se aplican a varios registros.
2. Relacionar estas tablas mediante una clave externa.

Una tabla 1NF estará en 2NF si y solo si, dada una clave primaria y cualquier atributo que no sea un constituyente de la clave primaria, el atributo no clave depende de **toda** la clave primaria en vez de solo una parte de ella.

Ejemplos:

Ejemplo1:

Esto se soluciona separando el atributo N_TRABAJADOR a una tabla separada

Tercera regla

1. Eliminar aquellos campos que no dependan de la clave.
2. Ninguna columna puede depender de una columna que no tenga una clave.
3. No puede haber datos derivados.

•

Ventajas de una base normalizada

- **Reduce la duplicación de datos:** Las bases de datos pueden contener una cantidad significativa de información, tal vez millones o miles de millones de piezas de datos. La normalización de una base de datos reduce su tamaño y evita la duplicación de datos. Se asegura de que cada pieza de datos se almacenen sólo una vez.
- **Grupos de datos lógicos:** Los desarrolladores de aplicaciones que crean aplicaciones para "hablar" con una base de datos les resulta más fácil tratar con una base de datos normalizada. Los datos que acceden está organizado de manera más lógica en una base de datos normalizada, a menudo similar a la forma en que los objetos del mundo real que representan los datos están organizados.
- **Mejora el tiempo de respuesta y velocidad:** Esta ventaja se menciona porque la base de datos se hace pequeña en algunos casos debido a que se eliminan los datos duplicados.
- **Mantenimiento rápido:** Menos índices por tablas aseguran un mantenimiento más rápido.
- **Consistencia de datos:** Los datos siempre son reales y no ambiguos.
- **Compactación de las tablas de la base de datos:** cuando se normaliza la base de datos , a veces se transforma una tabla muy grande en pequeñas tablas, la compactación significa tener el tamaño mínimo y requerido.
- **Reduce las anomalías de inserción , borrado y actualización :**
 - inserción: ocurre cuando queremos insertar y los datos no están completos o no están bien insertados en los atributos de destino.
 - borrado: ocurre cuando queremos eliminar y los datos no están completos o no están bien eliminados en los atributos correspondientes.
 - Actualización: ocurre cuando queremos actualizar y los datos no están completos o no están correctamente actualizados en los atributos de destino.

17.- Qué es un job.

Los Jobs son unidades de trabajo que pueden programarse para ejecutarse con el Job Manager(administrador de Jobs). Una vez que el Job ha sido terminado o completado su tarea, este puede ser programado para ejecutarse nuevamente (los Jobs son reusables).

Los Jobs tienen un estado el cual indica que están haciendo actualmente. Cuando se construyen , los job comienzan con un valor de **estado NONE**. Cuando un job esta programado para ejecutarse , entonces su estado cambia a **WAITING (en espera)**. Cuando un job comienza a correr , su estado cambia a **RUNNING**. Cuando la ejecución finaliza (de manera normal o a travez de una cancelación) el estado regresa a **NONE** (Ninguno).

Anexo

39. Norma INE

Es la herramienta informática utilizada para administrar y publicar el marco normativo vigente

40. Partidos Políticos

Son organizaciones de ciudadanos que compiten en las elecciones para integrar los órganos de gobierno y representación popular. Poseen un conjunto de documentos (plataforma política, programa de acción y estatutos) en los que expresan sus propuestas de gobierno, ideología y reglas de organización interna.

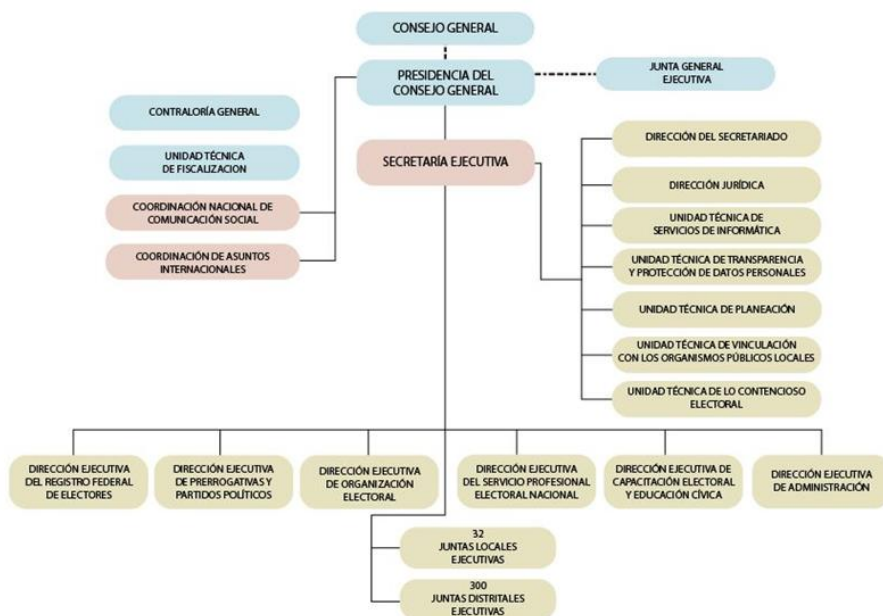
41. INE

El Instituto Nacional Electoral es el organismo público autónomo encargado de organizar las elecciones federales, es decir, la elección del Presidente de la República, Diputados y Senadores que integran el Congreso de la Unión, así como organizar, en coordinación con los organismos electorales de las entidades federativas, las elecciones locales en los estados de la República y el Distrito Federal.

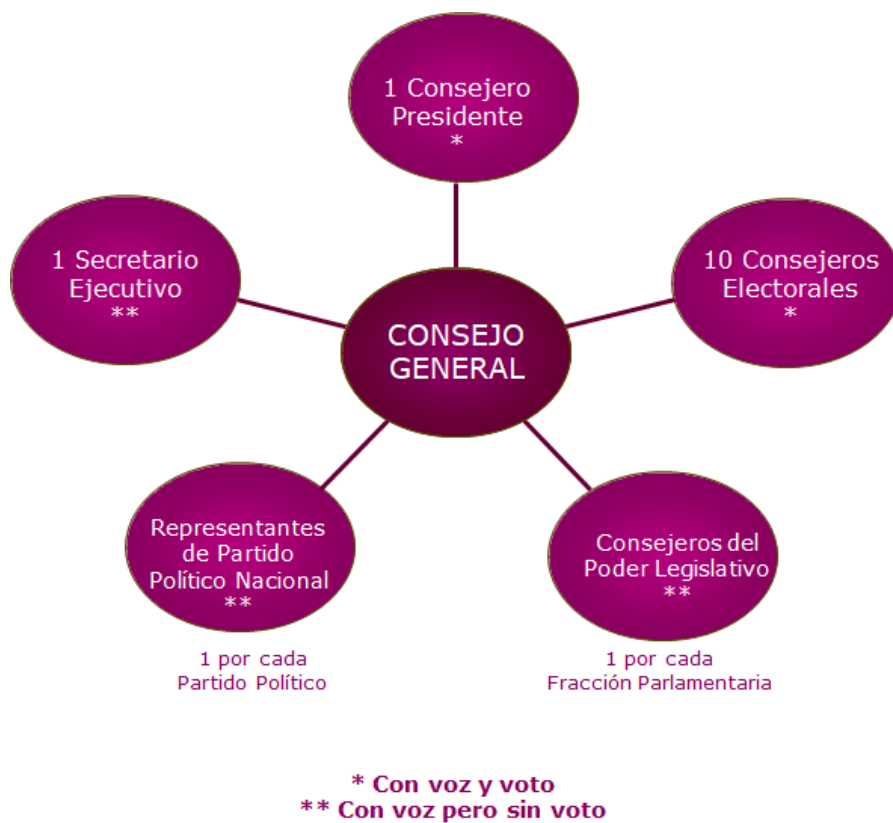
42. Consejo General

Es el órgano superior de dirección, Consejo responsable de vigilar el cumplimiento de las disposiciones constitucionales y legales en materia electoral, así como de velar porque los principios de certeza, legalidad, independencia, imparcialidad, máxima publicidad y objetividad guíen todas las actividades del Instituto.

43. Estructura del INE



44.- ESTRUCTURA DEL CONSEJO



45.- DIRECCIONES EJECUTIVAS

- DERFE - Dirección Ejecutiva del Registro Federal de Electores.
- DEPPP - Dirección Ejecutiva de Prerrogativas y Partidos Políticos.
- DEOE - Dirección Ejecutiva de Organización Electoral
- DESPE - Dirección Ejecutiva del Servicio Profesional Electoral Nacional.
- DECEYEC - Dirección Ejecutiva de Capacitación Electoral y Educación Cívica.
- DEA - Dirección Ejecutiva de Administración.

45.-Actividades principales UNICOM

- Elaborar y proponer al Secretario Ejecutivo los proyectos estratégicos en materia de informática que coadyuven al desarrollo de las actividades del Instituto, para su presentación ante el Comité de Tecnologías de la Información y Comunicaciones, la Junta y/o el Consejo, según corresponda.

- Apoyar a las diversas áreas del Instituto, en la optimización de sus procesos, mediante el desarrollo y/o la implementación de sistemas y servicios informáticos y de telecomunicaciones.

- Administrar y operar la infraestructura de cómputo y comunicaciones que soporta los servicios y sistemas que se encuentran disponibles a través de la Red Nacional de Informática, la cual permite comunicar a todo el personal que labora en el Instituto a nivel nacional, así como difundir información a la ciudadanía.

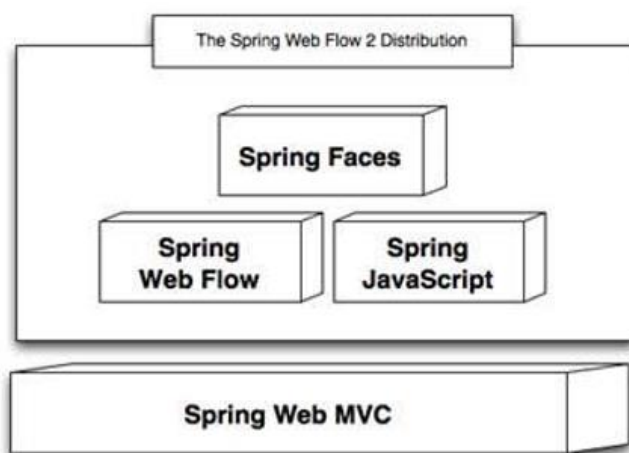
- Coordinar y ejecutar el desarrollo y operación del Programa de Resultados Electorales Preliminares (PREP), el cual permite difundir de forma inmediata a la ciudadanía, los resultados preliminares de las elecciones federales.

- Proporcionar capacitación, asesoría y soporte técnico a todo el personal del Instituto, en el uso de sistemas, servicios y equipos de cómputo.

46.- ¿Qué son los Organismos Públicos Locales (OPL)?

Son los encargados de la organización de las elecciones en su entidad federativa. A partir de 2014 el INE trabaja con los OPL a fin de homologar los estándares con los que se organizan los procesos electorales locales.

47 .- componentes de Spring web Flow



48 .- A que estructura de control pertenece el for??

ESTRUCTURA ITERATIVA O REPETITIVA

Permiten ejecutar de forma repetida un bloque específico de instrucciones. Las instrucciones se repiten mientras o hasta que se cumpla una determinada condición. Esta condición se conoce como

49.- ¿ cuando termina la ejecución de un hilo LA CLASE THREAD?

La forma más directa para hacer un programa multihilo es extender la clase Thread, y redefinir el método run(). Este método es invocado cuando se inicia el hilo (mediante una llamada al método start() de la clase Thread). El hilo se inicia con la llamada al método run() y termina cuando termina éste.

50.- Definición de primefaces

PrimeFaces es una biblioteca de componentes para JavaServer Faces (JSF) de código abierto que cuenta con un conjunto de componentes enriquecidos que facilitan la creación de las aplicaciones web.

51.- HQL

Hibernate utiliza un lenguaje de consulta potente (HQL) que se parece a SQL. Sin embargo, comparado con SQL, HQL es completamente orientado a objetos y comprende nociones como herencia, polimorfismo y asociación.

52 .- Características de javaBean

Un JavaBean debe cumplir las siguientes características para ser considerado como tal y conseguir que sean realmente componentes reutilizables.

1. Debe implementar la interfaz Serializable o la interfaz Externalizable.
Se suelen usar para encapsular varios objetos en uno, de manera que implementando alguna de estas interfaces se puede guardar el estado de cada objeto.
2. Debe tener un constructor vacío, sin argumentos, para instanciar un nuevo objeto de manera estándar.
3. Todas las variables de la instancia deben ser privadas.
4. Debe implementar los getters y setters de las variables o propiedades, los cuales serán públicos y son los que se utilizarán para acceder o actualizar las variables privadas.

53.- ¿ Para qué sirve un índice ?

Un índice sirve para buscar datos rápidamente, y no tener que recorrer toda la tabla secuencialmente en busca de alguna fila concreta

54.- Propiedades de la transacción ACID

- **Atómica (Atomic)** : Las transacciones son atómicas , significa que las declaraciones SQL contenidas en una sola transacción de trabajo.
- **Consistente (Consistent)** : Se asegura que las transacciones de los estados de la base de datos permanezca consistente. Significa que la base de datos está en un estado consistente cuando la transacción comienza y que al finalizar la transacción sigue en un estado consistente.
- **Aisladas (Isolate)**: Las transacciones deben estar separadas es decir no deben interferir unas con otras.
- **Durable** : Una vez que se ha hecho commit a una transacción , los cambios en la base de datos son preservados, incluso si en la máquina en donde esta la base de datos se bloquea más tarde.

55.- Aumentar el tamaño de un tableSpace

Mediante un archivo es

Para redimensionar un tablespace

- alter database datafile 'path_completo_del_data_file' resize 1024M;

Para agregar un tablespace

- alter tablespace nombre_del_tablespace
add datafile 'path_completo_del_data_file' size 1024M;

56.- Una pregunta incluía estos dos conceptos.ALL_OBJECTS Y USER_OBJECTS.

USER:OBJECTS .- lista solo los objetos que son propiedad del usuario actual.

ALL_OBJECT .- enumera todos los objetos que son propiedad del usuario actual y aquellos objetos a los que el usuario actual tiene acceso, es decir que no son solo de él sino que aquellos a los que tiene privilegios de ver.

57.- Operadores

Operador	Descripción
UNION ALL	Regresa todas las filas recuperadas por las consultas, incluyendo aquellas filas duplicadas.
UNION	Regresa todas las filas no duplicadas recuperadas por las consultas.
INTERSECT	Regresa filas que son recuperadas por ambas consultas.
MINUS	Regresa las filas restantes cuando las filas recuperadas con la segunda consulta son sustraídas de las filas recuperadas en la

	primera columna,
--	------------------

58.-¿cómo podrías representar la herencia múltiple?

R: class A extends B implements i1, i3

59 .- ¿ en que paquete de clases se encuentra File ?

se encuentra en la biblioteca java.io.File;

60 .- Tipos de recuperación de datos

Lazy

Al especificar una relación en un mapeo de Hibernate usaremos el atributo "lazy" para definir el "cuando".

Por defecto lazy es igual a true. Esto significa que la colección no se recupera de la base de datos hasta que se hace alguna operación sobre ella.

Si fijamos lazy a false, cuando se recupere la información de A también se traerá toda la información de los objetos relacionados de B (este era el comportamiento por defecto en la versión 2 de Hibernate).

Shell

```
1 <set name="bs" lazy="false">
```

Fetch

Por otro lado el atributo "fetch" define que SQLs se van a lanzar para recuperar la información.

Por defecto fetch es igual a select. Esto implica que para recuperar cada objeto de B se lanzará una nueva consulta a la base de datos.

Si fijamos el valor de fetch a join, en la misma consulta que se recupera la información de A también se recuperará la información de todos los objetos relacionados de B. Esto se consigue con un left outer join en la sentencia SQL.

Shell

```
1 <set name="bs" fetch="join">
```

1 <set name="bs" fetch="join">

61. POJO

POJO son las iniciales de “**Plain Old Java Object**”, que puede interpretarse como “Un objeto Java Plano Antiguo”. Un POJO es una instancia de una clase que no extiende ni implementa nada en especial

Un POJO no debe contener lo siguiente:

- Extender clases preespecificadas, Ej: `public class GFG extends javax.servlet.http.HttpServlet {...}` no es una clase POJO.
- Implementar interfaces preespecificadas, Ej: `public class Bar implements javax.ejb.EntityBean {...}` no es una clase POJO.
- Anotaciones preespecificadas, Ej: `@javax.persistence.Entity public class Baz {...}` no es una clase POJO.